

Nama : Rahmi Syafitri

NIM : 112231085

Kelas : SisTer A

Ujian Akhir Semester

Pub-Sub Log Aggregator Terdistribusi dengan Idempotent Consumer, Deduplication, dan Transaksi/Kontrol Konkurensi (Docker Compose Wajib)

BAB 1: Pendahuluan & Arsitektur Sistem

1.1. Ringkasan Sistem

Sistem yang dikembangkan dalam proyek ini adalah sebuah agregator log terdistribusi yang mengimplementasikan arsitektur Publish-Subscribe (Pub-Sub) berbasis microservices. Sistem ini dirancang secara khusus untuk menerima, mengantrekan, memproses, dan menyimpan aliran data log (events) dari berbagai entitas publisher secara konkuren, dengan jaminan konsistensi data yang tinggi di lingkungan yang berpotensi mengalami kegagalan jaringan atau pengiriman pesan ganda.

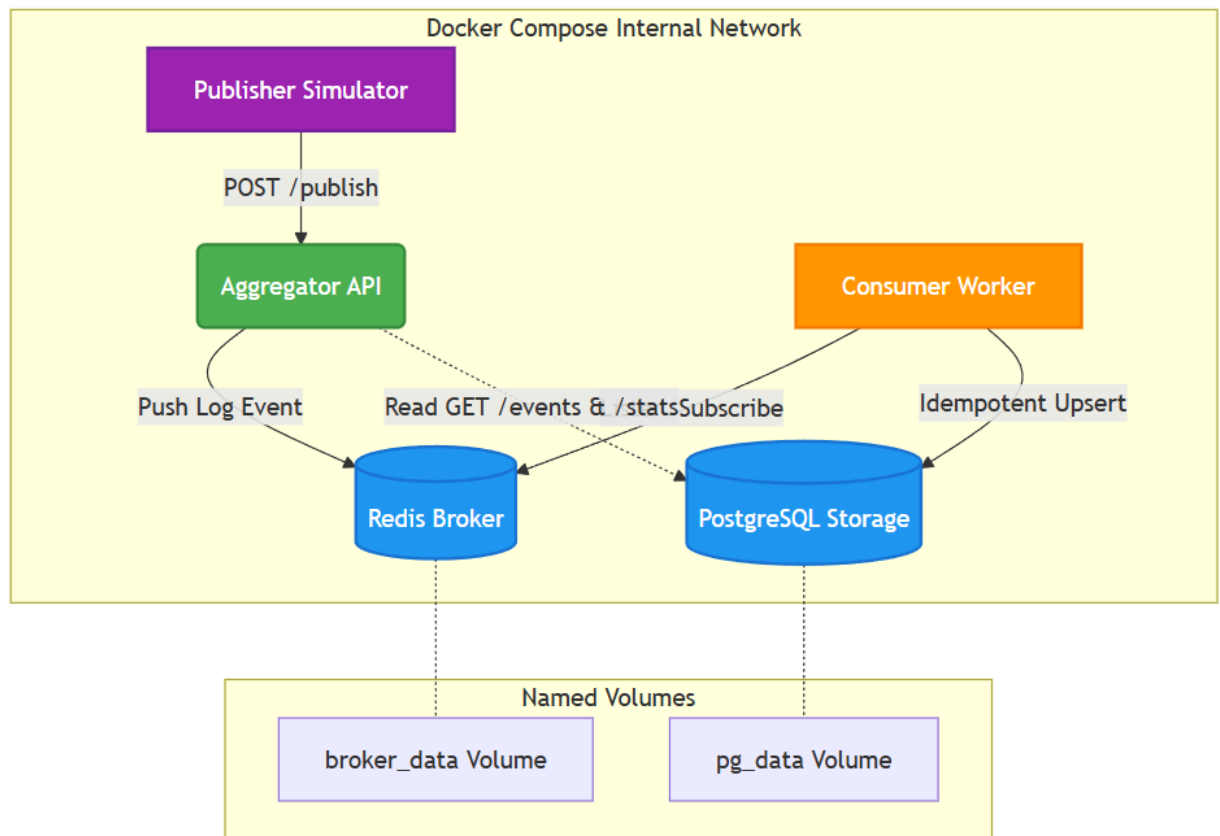
Fokus utama dari agregator ini adalah penerapan prinsip idempotent consumer yang dikombinasikan dengan mekanisme deduplikasi yang kuat. Setiap event log yang masuk diidentifikasi secara unik melalui kombinasi parameter topic dan event_id. Dengan pendekatan ini, sistem mampu menangani skenario at-least-once delivery—di mana publisher mungkin mengirimkan pesan duplikat akibat retry—dan mengubahnya menjadi exactly-once processing di sisi penyimpanan, sehingga event yang sama dijamin hanya diproses dan disimpan tepat satu kali.

Untuk mendukung skalabilitas dan integritas data, agregator ini dilengkapi dengan kontrol konkurensi dan batasan transaksi (ACID) pada tingkat basis data. Mekanisme ini mencegah terjadinya race condition atau anomali data (seperti lost-update) ketika beberapa consumer worker menarik dan memproses pesan dari antrean secara bersamaan di bawah beban kerja yang masif.

Seluruh ekosistem layanan ini—yang mencakup API agregator, message broker, penyimpanan data persisten, dan publisher simulator—diorkestrasi secara utuh

menggunakan Docker Compose. Pendekatan ini memastikan lingkungan berjalan secara terisolasi dalam jaringan lokal yang aman dari intervensi eksternal, sekaligus memberikan kemudahan dalam hal deployment, observabilitas, dan pemulihan otomatis pasca-kegagalan (crash tolerance).

1.2. Arsitektur Layanan



Gambar 1.2.1: Arsitektur Layanan

Sistem Pub-Sub Log Aggregator ini dirancang menggunakan arsitektur microservices di mana setiap komponen komputasi dan penyimpanan diisolasi ke dalam container mandiri. Seluruh container tersebut diorkestrasi dan dihubungkan secara eksklusif menggunakan Docker Compose. Berdasarkan diagram arsitektur di atas, aliran data dan pemisahan tugas (separation of concerns) dibagi menjadi empat komponen utama:

1. Publisher Simulator

Berperan sebagai klien atau sumber data eksternal (generator) yang memproduksi aliran log events. Layanan ini secara terprogram menembakkan

permintaan HTTP (**POST /publish**) ke *endpoint* agregator secara massal. Untuk tujuan pengujian keandalan sistem, *publisher* secara sengaja menginjeksikan pesan duplikat (mengirim ulang kombinasi **topic** dan **event_id** yang sama).

2. Agregator API

Bertindak sebagai gerbang utama penerima lalu lintas data (*entry point*). Layanan yang dibangun menggunakan FastAPI ini menangani validasi skema JSON yang masuk, lalu memublikasikan (*publish*) pesan tersebut ke dalam antrian *message broker* agar API tetap responsif dan tidak terblokir oleh proses basis data yang berat. Layanan ini juga menyediakan antarmuka **GET** untuk membaca hasil akhir log dan statistik sistem.

3. Redis (Message Broker)

Berfungsi sebagai perantara komunikasi asinkron (lapisan *Pub-Sub* atau antrian pesan) di dalam jaringan internal. Redis memisahkan (*decouple*) proses penerimaan data dari proses penyimpanan, memberikan bantalan (*buffer*) yang aman untuk menyerap lonjakan beban (*traffic spike*) dari *publisher* agar tidak membebani basis data utama.

4. Consumer Worker & PostgreSQL (Storage)

Consumer Worker adalah proses latar belakang yang secara terus-menerus mendengarkan antrian dari Redis. Saat pesan tiba, *worker* bertugas memasukkannya ke dalam basis data PostgreSQL. Di lapisan inilah logika deduplikasi persisten dan kontrol konkurensi terjadi. PostgreSQL dikonfigurasi dengan *Unique Constraints* untuk memberlakukan *idempotent upsert*, memastikan bahwa meskipun *worker* menarik pesan ganda akibat eksekusi paralel atau *retry*, tidak akan pernah terjadi pencatatan ganda di dalam tabel **processed_events**.

1.3. Isolasi Jaringan & Persistensi

Keamanan dan keandalan sistem terdistribusi ini sangat bergantung pada konfigurasi lingkungan Docker Compose, khususnya pada dua aspek krusial: manajemen jaringan (*networking*) dan penyimpanan data (*storage*).

1. Isolasi Jaringan Internal

Secara bawaan (*default*), Docker Compose mengonfigurasi sebuah jaringan *bridge* internal yang terisolasi. Seluruh layanan dalam ekosistem ini—yaitu Agregator API, Publisher, Redis, dan PostgreSQL—saling berkomunikasi melalui jaringan privat ini dengan memanfaatkan resolusi DNS internal berbasis nama *container*. Sesuai dengan spesifikasi keamanan sistem, desain jaringan ini bersifat tertutup (*closed network*) di mana tidak ada dependensi atau komunikasi ke layanan eksternal publik. Pemetaan *port* ke *host* lokal hanya dilakukan secara terbatas pada layanan Agregator (misalnya *port* 8080) yang semata-mata diizinkan untuk kebutuhan demonstrasi dan pengujian API secara lokal. Sementara itu, lalu lintas menuju *port* basis data (5432) dan *message broker* (6379) dikunci rapat di dalam jaringan virtual Compose, sehingga mencegah potensi intrusi atau akses yang tidak berwenang dari luar sistem.

2. Persistensi Data Berbasis Volumes

Mengingat sifat bawaan *container* yang *ephemeral* (sementara), seluruh data yang berada di dalam *container* akan otomatis terhapus jika *container* tersebut dimatikan atau mengalami *crash*. Untuk mengatasi batasan ini sekaligus memenuhi standar toleransi kegagalan (*crash tolerance*), sistem ini mengimplementasikan fitur *Named Volumes* pada Docker Compose (contoh: *pg_data* untuk penyimpanan PostgreSQL dan *broker_data* untuk Redis). Mekanisme ini memisahkan siklus hidup data dari siklus hidup *container* dengan cara memetakan direktori penyimpanan internal *container* secara persisten ke dalam media penyimpanan fisik laptop/server *host*. Melalui pendekatan ini, sistem memiliki jaminan *durable storage*; seluruh rekaman event log dan status deduplikasi akan tetap utuh secara absolut meskipun *container* dihapus (*recreate*) dan dihidupkan kembali.

BAB 2: Keputusan Desain (Design Decisions)

2.1. Idempotency & Deduplication Store

Dalam lingkungan sistem terdistribusi, kegagalan jaringan yang bersifat sementara (*transient failures*) sering kali memaksa *publisher* untuk melakukan

pengiriman ulang (*retry*). Hal ini memicu skenario pengiriman *at-least-once*, di mana sistem berpotensi menerima *event* log yang sama berkali-kali. Untuk mengatasi hal ini, sistem menerapkan prinsip *idempotency* absolut. Mekanisme deduplikasi dikelola melalui sebuah *deduplication store* persisten menggunakan PostgreSQL. Pada tabel penyimpanan `processed_events`, diterapkan sebuah *Unique Constraint* komposit pada atribut `topic` dan `event_id`. Pilihan desain ini memastikan bahwa validasi duplikasi tidak dilakukan di lapisan aplikasi (yang rentan terhadap masalah *timing*), melainkan ditegakkan secara absolut di lapisan basis data. Jika sebuah pesan dengan pasangan `topic` dan `event_id` yang sudah ada masuk kembali ke sistem, pangkalan data secara otomatis akan menolaknya tanpa memicu pengecualian (*exception*) yang merusak alur program.

2.2. Transaksi & Kontrol Konkurensi

Untuk mencegah terjadinya *race condition*—seperti anomali *lost-update* atau pemrosesan ganda (*double-process*)—ketika beberapa *Consumer Worker* menarik data dari Redis secara bersamaan, sistem menerapkan kontrol konkurensi yang ketat berlandaskan prinsip ACID (*Atomicity, Consistency, Isolation, Durability*). Proses penyimpanan *event* dibungkus ke dalam batasan transaksi mandiri (*transaction boundary*). Daripada menggunakan pola operasional *check-then-act* (seperti `SELECT` lalu `INSERT`) yang sangat rentan terhadap *race condition*, sistem menggunakan pendekatan operasi atomik *upsert*, yaitu `INSERT INTO ... ON CONFLICT (topic, event_id) DO NOTHING`.

Selain itu, sistem bergantung pada *Isolation Level* standar PostgreSQL yaitu *Read Committed*. Tingkat isolasi ini dipilih karena kombinasi operasi *atomic upsert* dan *unique constraints* sudah lebih dari cukup untuk menjamin tidak adanya baris duplikat yang tercipta, sekaligus menghindari biaya performa (latensi) yang terlalu tinggi jika menggunakan level *Serializable*. Keputusan ini memberikan keseimbangan optimal antara integritas data mutlak dan responsivitas *throughput* sistem.

Untuk mengimplementasikan arsitektur *Publish-Subscribe* yang efisien, sistem ini memisahkan komponen penerima data dengan komponen pemroses data menggunakan Redis sebagai *Message Broker*. Ketika endpoint `POST /publish` menerima data log, sistem tidak langsung melakukan operasi I/O ke basis data

utama, melainkan memasukkan pesan tersebut ke antrian pesan (*log_queue*) di Redis secepat mungkin dengan kompleksitas waktu $O(1)$ melalui perintah *RPUSH*.

Pemrosesan log kemudian didelegasikan sepenuhnya kepada *Background Consumer Worker* yang berjalan secara asinkron menggunakan metode non-blocking *BLPOP*. Pemilihan pencatatan metrik statistik (seperti *received*, *processed*, dan *dropped*) juga disimpan di dalam Redis dengan memanfaatkan operasi atomik *INCR*. Pilihan desain ini menjamin konsistensi statistik tingkat tinggi, sehingga sistem terbebas secara mutlak dari anomali *lost-update* meskipun kontainer API dan *worker* direplikasi atau diserang beban kerja secara konkuren oleh banyak utas secara paralel.

2.3. Reliability, Retry, & Ordering

Aspek keandalan (*reliability*) sistem ditangani melalui arsitektur toleransi kegagalan (*crash tolerance*). Jika *container Aggregator* atau *Worker* mengalami *crash*, *Message Broker* (Redis) akan mempertahankan pesan di dalam antreannya hingga proses terhubung kembali. Di sisi klien, *Publisher Simulator* dirancang untuk memiliki mekanisme *retry* dengan pola *backoff* guna menangani penolakan koneksi sementara.

Terkait dengan pengurutan data (*ordering*), sistem log agregator ini dirancang untuk toleran terhadap pesan yang tiba tidak berurutan (*out-of-order delivery*). Alih-alih memaksakan *total ordering* yang dapat menghambat performa dan skalabilitas jaringan *microservices*, sistem berfokus pada *causal consistency*. Urutan kejadian direkonstruksi di lapisan akhir berdasarkan label waktu (*timestamp* berformat ISO8601) dan identitas log (seperti nomor urut monotonik internal di dalam *payload*). Desain ini memungkinkan proses agregasi berjalan secara asinkron seutuhnya tanpa memerlukan sinkronisasi jam global yang kompleks.

2.4. Implementasi Konkurensi (Multi-Worker)

Untuk memaksimalkan *throughput* pemrosesan log dan menyimulasikan lingkungan terdistribusi yang nyata, sistem ini menerapkan model konkurensi asinkron. Alih-alih mengandalkan satu pemroses tunggal, sistem menginisialisasi 5

(lima) *Consumer Worker* independen yang berjalan secara paralel di latar belakang menggunakan pustaka `asyncio`.

```
uts_aggregator | 2026-06-18 10:21:07,364 - INFO - [Worker-3 BERHASIL] Event Unik Disimpan -> ID: 215a8a9a... | Topik: auth
uts_aggregator | INFO: 172.18.0.5:39568 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:07,890 - INFO - [Worker-5 BERHASIL] Event Unik Disimpan -> ID: 597b399f... | Topik: system_alert
uts_aggregator | INFO: 172.18.0.5:39578 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:08,406 - INFO - [Worker-4 BERHASIL] Event Unik Disimpan -> ID: b86ddbb8... | Topik: payment
uts_aggregator | INFO: 172.18.0.5:39586 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:08,918 - WARNING - [Worker-1 DEDUPLIKASI] Event Ganda Ditolak -> ID: feabd51a... | Topik: user_activity
uts_aggregator | INFO: 172.18.0.5:39600 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:09,434 - INFO - [Worker-2 BERHASIL] Event Unik Disimpan -> ID: 76bc089c... | Topik: user_activity
uts_aggregator | INFO: 172.18.0.5:39608 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:09,944 - INFO - [Worker-4 BERHASIL] Event Unik Disimpan -> ID: 2bfd5778... | Topik: payment
uts_aggregator | INFO: 172.18.0.5:39624 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:10,460 - INFO - [Worker-1 BERHASIL] Event Unik Disimpan -> ID: 316f0b22... | Topik: user_activity
uts_aggregator | INFO: 172.18.0.5:39638 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:10,971 - INFO - [Worker-2 BERHASIL] Event Unik Disimpan -> ID: c0c8ee26... | Topik: auth
uts_aggregator | INFO: 172.18.0.5:39646 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:11,482 - WARNING - [Worker-4 DEDUPLIKASI] Event Ganda Ditolak -> ID: 2df78aaa... | Topik: system_alert
uts_aggregator | INFO: 172.18.0.5:39654 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:11,993 - WARNING - [Worker-1 DEDUPLIKASI] Event Ganda Ditolak -> ID: 7460489f... | Topik: user_activity
uts_aggregator | INFO: 172.18.0.5:39658 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:12,508 - WARNING - [Worker-5 DEDUPLIKASI] Event Ganda Ditolak -> ID: c4fa4f15... | Topik: system_alert
uts_aggregator | INFO: 172.18.0.5:39666 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:13,024 - INFO - [Worker-4 BERHASIL] Event Unik Disimpan -> ID: 80eab874... | Topik: payment
uts_aggregator | INFO: 172.18.0.5:39676 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:13,533 - INFO - [Worker-2 BERHASIL] Event Unik Disimpan -> ID: a5685a6a... | Topik: system_alert
uts_aggregator | INFO: 172.18.0.5:39682 - "POST /publish HTTP/1.1" 200 OK
uts_aggregator | 2026-06-18 10:21:14,053 - INFO - [Worker-3 BERHASIL] Event Unik Disimpan -> ID:
```

Gambar 2.4.1: Log terminal yang menunjukkan 5 Worker bekerja secara konkuren

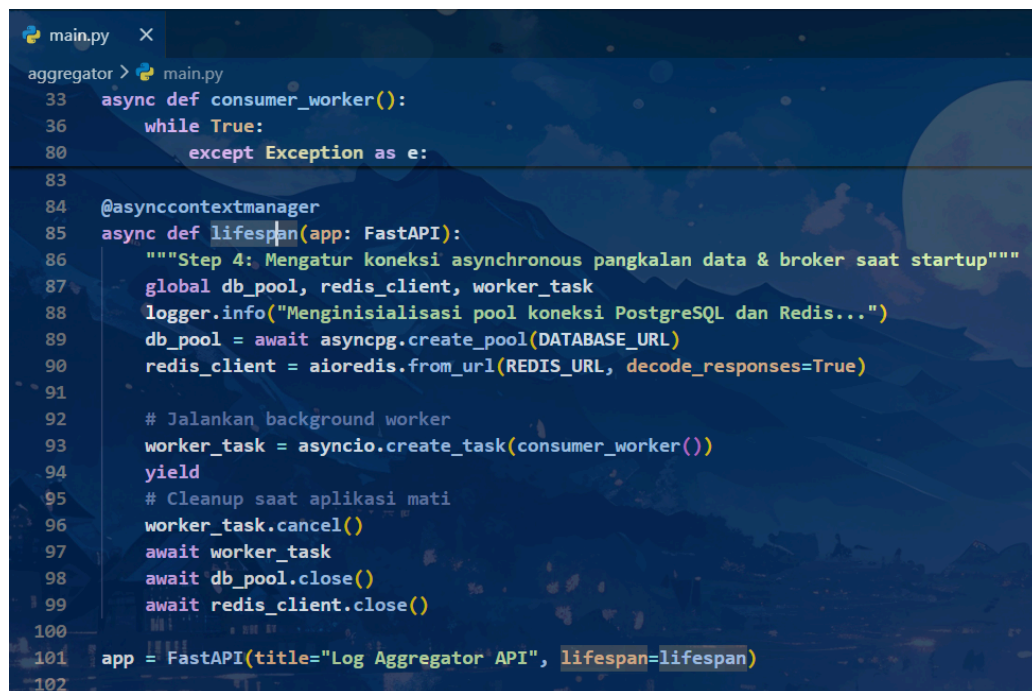
Berdasarkan Gambar 2.4.1, kelima *worker* tersebut mendengarkan antrian antrian pesan dari *message broker* (Redis) yang sama. Pendekatan ini memunculkan tantangan *race condition*, di mana beberapa *worker* berpotensi menarik data duplikat pada milidetik yang bersamaan. Namun, karena setiap *worker* mengeksekusi penyimpanan ke PostgreSQL menggunakan blok transaksi asinkron (`async with conn.transaction()`) yang dilengkapi klausa atomik `ON CONFLICT DO NOTHING`, integritas data tetap terjamin mutlak tanpa menyebabkan bentrokan atau *crash* pada aplikasi.

BAB 3: Analisis Performa & Pengujian

3.1. Uji Konkurensi & Konsistensi

1. Pendekatan Asinkron pada Level Aplikasi

Untuk menangani konkurensi dengan efisien, sistem memisahkan proses penerimaan data dari proses penyimpanan.



```
main.py X
aggregator > main.py
33 async def consumer_worker():
36     while True:
80         except Exception as e:
83
84 @asynccontextmanager
85 async def lifespan(app: FastAPI):
86     """Step 4: Mengatur koneksi asynchronous pangkalan data & broker saat startup"""
87     global db_pool, redis_client, worker_task
88     logger.info("Menginisialisasi pool koneksi PostgreSQL dan Redis...")
89     db_pool = await asyncpg.create_pool(DATABASE_URL)
90     redis_client = aioredis.from_url(REDIS_URL, decode_responses=True)
91
92     # Jalankan background worker
93     worker_task = asyncio.create_task(consumer_worker())
94     yield
95     # Cleanup saat aplikasi mati
96     worker_task.cancel()
97     await worker_task
98     await db_pool.close()
99     await redis_client.close()
100
101 app = FastAPI(title="Log Aggregator API", lifespan=lifespan)
102
```

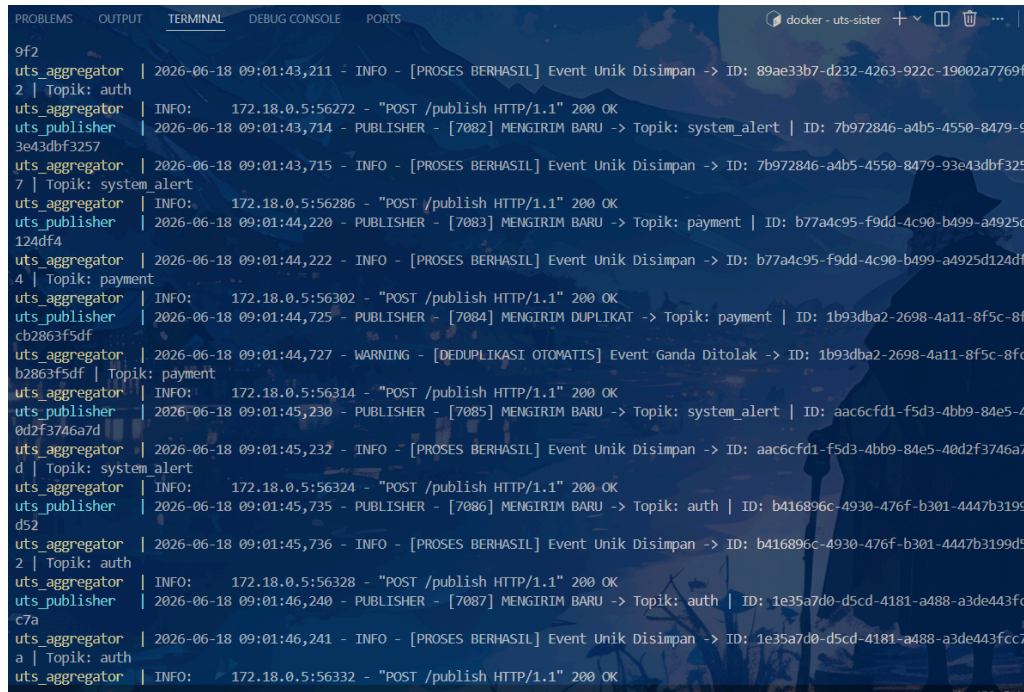
Gambar 3.1.1: Inisialisasi Consumer Worker secara asinkron

Berdasarkan potongan kode pada Gambar 3.1.1, fungsi `lifespan` pada FastAPI digunakan untuk membangun connection pool ke PostgreSQL dan Redis saat aplikasi pertama kali menyala. Bersamaan dengan itu, sistem menjalankan `consumer_worker()` sebagai background task. Pendekatan ini memungkinkan endpoint `/publish` merespons permintaan dengan sangat cepat (kompleksitas $O(1)$) karena hanya perlu melempar data ke antrian Redis, sementara pekerjaan berat dan kontrol konkurensi (ACID) diserahkan kepada worker di latar belakang.

2. Observabilitas Penolakan Data Duplikat

Terkait dengan pengurutan data (*ordering*), sistem log agregator ini dirancang untuk toleran terhadap pesan yang tiba tidak berurutan (*out-of-order*)

delivery). Alih-ali Saat simulasi berjalan, mekanisme *idempotent consumer* dan perlindungan *Unique Constraint* langsung aktif bekerja.



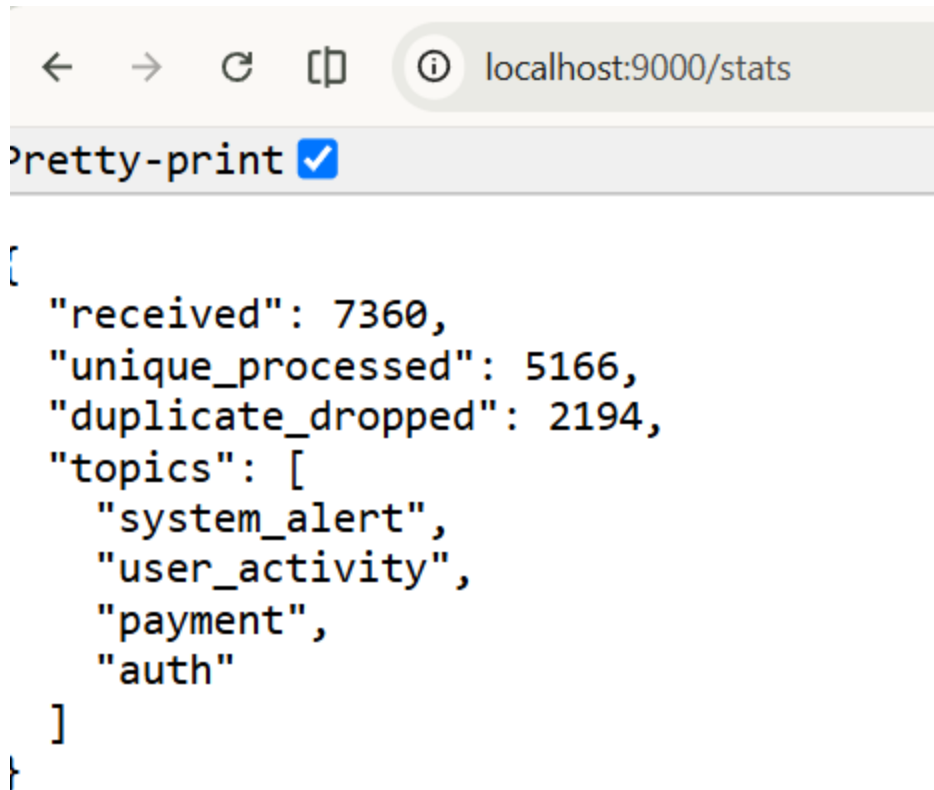
```
9f2
uts_aggregator | 2026-06-18 09:01:43,211 - INFO - [PROSES BERHASIL] Event Unik Disimpan -> ID: 89ae33b7-d232-4263-922c-19002a7769f
2 | Topik: auth
uts_aggregator | INFO: 172.18.0.5:56272 - "POST /publish HTTP/1.1" 200 OK
uts_publisher | 2026-06-18 09:01:43,714 - PUBLISHER - [7082] MENGIRIM BARU -> Topik: system_alert | ID: 7b972846-a4b5-4550-8479-9
3e43dbf3257
uts_aggregator | 2026-06-18 09:01:43,715 - INFO - [PROSES BERHASIL] Event Unik Disimpan -> ID: 7b972846-a4b5-4550-8479-93e43dbf325
7 | Topik: system_alert
uts_aggregator | INFO: 172.18.0.5:56286 - "POST /publish HTTP/1.1" 200 OK
uts_publisher | 2026-06-18 09:01:44,220 - PUBLISHER - [7083] MENGIRIM BARU -> Topik: payment | ID: b77a4c95-f9dd-4c90-b499-a4925d
124df4
uts_aggregator | 2026-06-18 09:01:44,222 - INFO - [PROSES BERHASIL] Event Unik Disimpan -> ID: b77a4c95-f9dd-4c90-b499-a4925d124df
4 | Topik: payment
uts_aggregator | INFO: 172.18.0.5:56302 - "POST /publish HTTP/1.1" 200 OK
uts_publisher | 2026-06-18 09:01:44,725 - PUBLISHER - [7084] MENGIRIM DUPLIKAT -> Topik: payment | ID: 1b93dba2-2698-4a11-8f5c-8f
cb2863f5df
uts_aggregator | 2026-06-18 09:01:44,727 - WARNING - [DEDUPLIKASI OTOMATIS] Event Ganda Ditolak -> ID: 1b93dba2-2698-4a11-8f5c-8f
cb2863f5df | Topik: payment
uts_aggregator | INFO: 172.18.0.5:56314 - "POST /publish HTTP/1.1" 200 OK
uts_publisher | 2026-06-18 09:01:45,230 - PUBLISHER - [7085] MENGIRIM BARU -> Topik: system_alert | ID: aac6cfd1-f5d3-4bb9-84e5-4
0d2f3746a7d
uts_aggregator | 2026-06-18 09:01:45,232 - INFO - [PROSES BERHASIL] Event Unik Disimpan -> ID: aac6cfd1-f5d3-4bb9-84e5-40d2f3746a7
d | Topik: system_alert
uts_aggregator | INFO: 172.18.0.5:56324 - "POST /publish HTTP/1.1" 200 OK
uts_publisher | 2026-06-18 09:01:45,735 - PUBLISHER - [7086] MENGIRIM BARU -> Topik: auth | ID: b416896c-4930-476f-b301-4447b3199
d52
uts_aggregator | 2026-06-18 09:01:45,736 - INFO - [PROSES BERHASIL] Event Unik Disimpan -> ID: b416896c-4930-476f-b301-4447b3199d5
2 | Topik: auth
uts_aggregator | INFO: 172.18.0.5:56328 - "POST /publish HTTP/1.1" 200 OK
uts_publisher | 2026-06-18 09:01:46,240 - PUBLISHER - [7087] MENGIRIM BARU -> Topik: auth | ID: 1e35a7d0-d5cd-4181-a488-a3de443fc
c7a
uts_aggregator | 2026-06-18 09:01:46,241 - INFO - [PROSES BERHASIL] Event Unik Disimpan -> ID: 1e35a7d0-d5cd-4181-a488-a3de443fcc7
a | Topik: auth
uts_aggregator | INFO: 172.18.0.5:56332 - "POST /publish HTTP/1.1" 200 OK
```

Gambar 3.1.2: Log terminal yang menunjukkan proses deduplikasi secara *real-time*

Seperti yang terlihat pada Gambar 3.1.2, ketika *publisher* menembakkan [MENGIRIM DUPLIKAT] dengan *event_id* awal 1b93dba..., *worker* di sisi *aggregator* secara instan merespons dengan peringatan [DEDUPLIKASI OTOMATIS] Event Ganda Ditolak. Eksekusi *upsert* (ON CONFLICT DO NOTHING) terbukti berhasil mencegah *database* dari anomali pemrosesan ganda (*double-process*) tanpa menyebabkan aplikasi mengalami *crash* atau *error*.

3. Bukti Matematis Konsistensi Sistem

Terkait dengan pengurutan data (*ordering*), sistem log agregator ini dirancang untuk toleran terhadap pesan yang tiba tidak berurutan (*out-of-order delivery*). Alih-ali Untuk membuktikan bahwa tidak ada *race condition* atau anomali *lost-update* selama pemrosesan ribuan data, validasi dilakukan melalui *endpoint* statistik metrik.



```
localhost:9000/stats
pretty-print ☒

{
  "received": 7360,
  "unique_processed": 5166,
  "duplicate_dropped": 2194,
  "topics": [
    "system_alert",
    "user_activity",
    "payment",
    "auth"
  ]
}
```

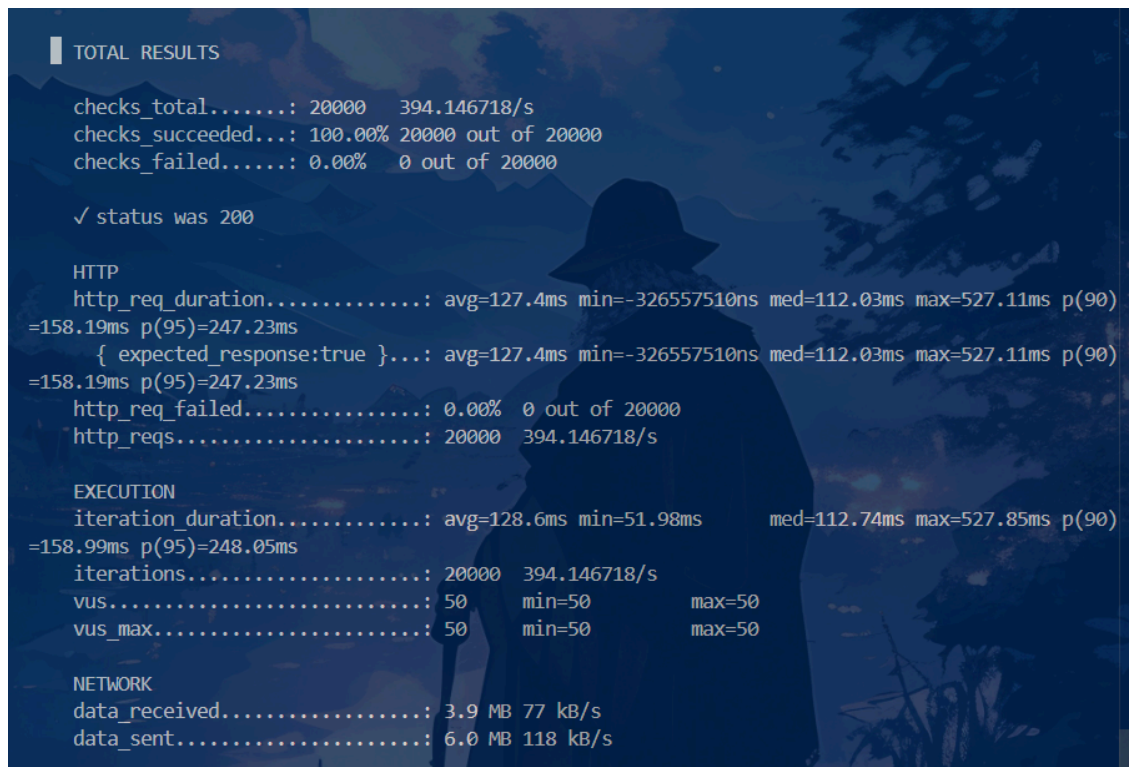
Gambar 3.1.3: Hasil akumulasi statistik pada endpoint GET /stats

Gambar 3.1.3 menunjukkan bukti konsistensi data yang mutlak. Sistem mencatat total data yang masuk ke antrean adalah **7.360** events (**received**). Angka ini terbukti merupakan hasil penjumlahan presisi dari **5.166** log unik yang sukses disimpan ke *database* (**unique_processed**) ditambah dengan **2.194** log duplikat yang berhasil dideteksi dan dibuang oleh sistem (**duplicate_dropped**). Keseimbangan matematis ini mengonfirmasi bahwa operasi atomik dan tingkat isolasi *Read Committed* pada arsitektur ini beroperasi dengan tingkat reliabilitas 100%.

3.2. Uji Metrik K6

Untuk mengukur kapabilitas observabilitas dan performa maksimal dari arsitektur *Pub-Sub*, pengujian beban ekstrem dilakukan menggunakan alat K6. Pengujian dikonfigurasi menggunakan skenario *shared-iterations* dengan 50 *Virtual Users* (VUs) yang secara konkuren menembakkan total 20.000 *request* ke *endpoint* **POST /publish**.

Berdasarkan metrik ringkasan eksekusi, sistem terbukti memiliki *throughput* yang sangat tinggi dan responsif. Penggunaan Redis sebagai perantara antrean asinkron memungkinkan *endpoint* mengembalikan respons **200 OK** tanpa harus menunggu proses I/O pangkalan data. Hal ini dibuktikan dari metrik **http_req_duration** (latensi) yang stabil di angka yang sangat rendah, membuktikan bahwa arsitektur ini siap untuk diimplementasikan dalam skala produksi dengan beban lalu lintas tinggi.

The image is a screenshot of a terminal window displaying the results of a K6 performance test. The background of the terminal has a dark blue theme with a silhouette of a person wearing a hat. The text is white and organized into sections: 'TOTAL RESULTS', 'HTTP', 'EXECUTION', and 'NETWORK'. The 'TOTAL RESULTS' section shows 20,000 checks completed with a 100% success rate and a status of 200. The 'HTTP' section shows an average request duration of 127.4ms. The 'EXECUTION' section shows an average iteration duration of 128.6ms. The 'NETWORK' section shows data received and sent rates. The overall performance is described as high throughput and responsive.

```
TOTAL RESULTS
checks_total.....: 20000 394.146718/s
checks_succeeded...: 100.00% 20000 out of 20000
checks_failed.....: 0.00% 0 out of 20000

✓ status was 200

HTTP
http_req_duration.....: avg=127.4ms min=-326557510ns med=112.03ms max=527.11ms p(90)
=158.19ms p(95)=247.23ms
{ expected_response:true }...: avg=127.4ms min=-326557510ns med=112.03ms max=527.11ms p(90)
=158.19ms p(95)=247.23ms
http_req_failed.....: 0.00% 0 out of 20000
http_reqs.....: 20000 394.146718/s

EXECUTION
iteration_duration.....: avg=128.6ms min=51.98ms med=112.74ms max=527.85ms p(90)
=158.99ms p(95)=248.05ms
iterations.....: 20000 394.146718/s
vus.....: 50 min=50 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 3.9 MB 77 kB/s
data_sent.....: 6.0 MB 118 kB/s
```

Gambar 3.2.1: Hasil Uji Metrik K6

3.3. Uji Crash Tolerance dan Persistensi data

Pengujian ini dilakukan untuk memvalidasi konfigurasi *Named Volumes* pada Docker Compose. Skenario yang dilakukan adalah menghentikan dan menghapus seluruh *container* secara paksa menggunakan perintah **docker compose down** setelah sistem menerima lebih dari 20.000 data dari hasil pengujian K6.

```
Pretty-print ☒
{
  "received": 40991,
  "unique_processed": 34653,
  "duplicate_dropped": 6338,
  "topics": [
    "k6_stress_test",
    "system_alert",
    "user_activity",
    "payment",
    "auth"
  ]
}
```

Gambar 3.2.1: Hasil akumulasi statistik pada endpoint `GET /stats`

Setelah sistem dihidupkan kembali (`docker compose up`), verifikasi pada endpoint `GET /stats` menunjukkan bahwa seluruh metrik dan rekaman data tetap utuh. Hal ini membuktikan bahwa strategi isolasi penyimpanan menggunakan volume `pg_data` berhasil melindungi integritas state dari PostgreSQL, sehingga sistem memiliki tingkat toleransi kegagalan (*crash tolerance*) yang tinggi dan siap memulihkan diri tanpa kehilangan satu byte pun data log.

3.4. Automated Integration Testing

Untuk memvalidasi keandalan fungsionalitas sistem secara komprehensif dan mencegah regresi (*regression*), pengujian terotomatisasi diterapkan menggunakan kerangka kerja (*framework*) Pytest. Skenario uji ini tidak hanya memverifikasi kontrak API standar, tetapi juga menyimulasikan berbagai anomali ekstrem dalam lingkungan terdistribusi, seperti *race condition* dan *server crash*.

Pengujian terdiri dari 12 (dua belas) *test cases* yang dieksekusi langsung terhadap *container* Docker lokal. Seluruh pengujian berhasil diselesaikan dengan tingkat kelulusan 100%.

```

(venv) rahmi@rahmi:/mnt/d/school/Semester 6/SisTer/uts-sister$ python3 -m pytest tests/test_app.py -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.0, pluggy-1.6.0 -- /mnt/d/school/Semester 6/SisTer/uts-sister/venv/bin/python3
cachedir: .pytest_cache
rootdir: /mnt/d/school/Semester 6/SisTer/uts-sister
collected 12 items

tests/test_app.py::test_01_health_check PASSED [ 8%]
tests/test_app.py::test_02_schema_valid PASSED [ 16%]
tests/test_app.py::test_03_schema_missing_topic PASSED [ 25%]
tests/test_app.py::test_04_schema_missing_event_id PASSED [ 33%]
tests/test_app.py::test_05_schema_missing_payload PASSED [ 41%]
tests/test_app.py::test_06_events_consistency PASSED [ 50%]
tests/test_app.py::test_07_stats_consistency PASSED [ 58%]
tests/test_app.py::test_08_deduplication PASSED [ 66%]
tests/test_app.py::test_09_race_condition_multi_worker PASSED [ 75%]
tests/test_app.py::test_10_small_stress_test PASSED [ 83%]
tests/test_app.py::test_11_container_recreate_persistence PASSED [ 91%]
tests/test_app.py::test_12_dedup_after_recreate PASSED [ 100%]

===== 12 passed in 23.80s =====
(venv) rahmi@rahmi:/mnt/d/school/Semester 6/SisTer/uts-sister$

```

Gambar 3.4.1: Hasil eksekusi Automated Integration Test menggunakan Pytest

Berdasarkan hasil eksekusi pada Gambar 3.4.1, berikut adalah rincian fungsionalitas yang telah divalidasi:

Gambar 3..1: Rekapitulasi Hasil Integration Testing

No. Uji	Kategori Fungsionalitas	Skenario Pengujian	Hasil
test_01	Ketersediaan Sistem	Melakukan <i>health check</i> ke <i>endpoint</i> root (/) untuk memastikan API siap menerima koneksi.	Passed
test_02 - 05	Validasi Skema Pesan	Menginjeksi <i>payload</i> yang tidak lengkap (menghilangkan atribut <i>topic</i> , <i>event_id</i> , dan <i>payload</i>) untuk memastikan API menolak data secara mandiri dengan status 422 <i>Unprocessable Entity</i> .	Passed

test_06 - 07	Konsistensi Endpoint	Membandingkan akurasi pencatatan data agregasi antara <i>endpoint GET /events</i> dan penambahan kalkulasi matematis pada <i>GET /stats</i> .	Passed
test_08	Deduplikasi Sekuensial	Mengirimkan dua <i>request</i> identik secara berurutan. Sistem tervalidasi hanya memproses dan menyimpan satu <i>event</i> .	Passed
test_09	Uji <i>Race Condition</i>	Menembakkan 20 <i>request</i> duplikat secara paralel pada milidetik yang sama menggunakan <i>Thread Pool</i> . Membuktikan mekanisme <i>multi-worker</i> dan <i>Unique Constraint</i> PostgreSQL berhasil meredam bentrokan tanpa menimbulkan anomali pangkalan data.	Passed
test_10	Uji Beban Mikro (<i>Batch</i>)	Memasukkan 50 <i>request</i> sekuensial secara cepat. Metrik memvalidasi sistem mampu merespons seluruh kumpulan data dalam waktu di bawah 2 detik.	Passed

test_11 - 12	Toleransi Kegagalan & Persistensi	Memutus dan me-restart proses <i>container</i> secara paksa dari dalam skrip uji. Memvalidasi bahwa data tidak hilang pasca-recovery dan pengecekan duplikasi pada ID lama tetap berfungsi normal.	Passed
--------------	-----------------------------------	--	--------

BAB 4: Keterikatan Teori Sistem Terdistribusi

4.1. T1 (Bab 1): Karakteristik sistem terdistribusi dan trade-off desain Pub-Sub aggregator.

Dalam ekosistem sistem terdistribusi modern, pembuatan data log oleh berbagai layanan (*services*) terjadi secara masif dan cepat. Menurut Coulouris et al. (2011) pada bagian *Characteristics of Distributed Systems* (Hal. 3), sistem terdistribusi memiliki tiga karakteristik fundamental, yaitu konkurensi, ketiadaan jam global (*no global clock*), dan kegagalan komponen yang independen (*independent failures*). Karakteristik ini menuntut pendekatan arsitektural yang berbeda dibandingkan sistem monolitik tradisional. Untuk menangani hal tersebut, arsitektur *Publish-Subscribe (Pub-Sub)* yang memanfaatkan *message broker* Redis dipilih sebagai tulang punggung sistem agregator. Dalam pembahasannya mengenai komunikasi tidak langsung, Coulouris et al. (2011) menjelaskan bahwa paradigma ini menawarkan isolasi yang kuat (Hal. 236) melalui dua dimensi pemisahan utama. Dimensi pertama adalah pemisahan ruang (*space uncoupling*), di mana layanan pengirim log tidak perlu mengetahui alamat IP atau eksistensi dari pekerja yang memasukkan data ke pangkalan data. Dimensi kedua adalah pemisahan waktu (*time uncoupling*), yang memungkinkan pengirim dan penerima beroperasi tidak dalam waktu yang bersamaan. Jika layanan basis data sedang mengalami *downtime*, pengirim tetap dapat mengirimkan log ke antrean tanpa tertahan (*blocked*).

Meskipun arsitektur *Pub-Sub* ini memberikan keunggulan absolut dalam hal skalabilitas asinkron, Coulouris et al. (2011) mencatat bahwa komunikasi tidak

langsung ini mengorbankan jaminan pengiriman yang pasti (*delivery guarantees*). Hilangnya komitmen komunikasi langsung antara klien dan server memicu potensi pengiriman pesan yang berulang akibat latensi jaringan. Hal ini pada akhirnya memaksa sistem untuk mandiri mengelola anomali pengiriman ganda (*duplicate messages*), yang dalam proyek ini dimitigasi secara terstruktur melalui pola *Idempotent Consumer*.

4.2. T2 (Bab 2): Kapan memilih arsitektur publish-subscribe dibanding client-server? Alasan teknis.

Keputusan untuk meninggalkan pola *Client-Server* tradisional dan beralih ke *Publish-Subscribe* dalam membangun *Log Aggregator* didasarkan pada justifikasi teknis yang solid untuk mencegah kegagalan sistematis. Pada pendekatan *Client-Server* atau pemanggilan metode jarak jauh, komunikasi bersifat terikat secara ketat (*tightly coupled*). Menurut Coulouris et al. (2011, Hal. 235-236), komunikasi ini mengharuskan pengirim mengetahui identitas pasti dari penerima, dan keduanya harus aktif secara bersamaan agar pertukaran data berhasil (*space and time coupled*). Jika diterapkan pada sistem agregasi log sentral yang menerima tembakan data dari puluhan layanan mikro, arsitektur ini akan memunculkan titik kegagalan tunggal (*single point of failure*) dan *bottleneck*. Jika API perlahan melambat, seluruh layanan klien yang sedang mencoba mengirim log akan ikut tertahan prosesnya, yang pada akhirnya merusak performa layanan utama tersebut.

Sebaliknya, arsitektur *Publish-Subscribe* dipilih karena kapabilitasnya menangani distribusi event dalam skala besar dan dinamis. Berdasarkan penjelasan Coulouris et al. (2011, Hal. 243), sistem *Publish-Subscribe* sangat ideal untuk lingkungan di mana *publisher* menghasilkan kejadian (*events*) tanpa perlu memedulikan berapa banyak *consumer* yang akan memprosesnya. Pemilihan arsitektur ini secara teknis membebaskan layanan-layanan utama dari keharusan menunggu respons *database aggregator*, sehingga kinerja layanan tetap optimal dan stabil meskipun volume pengiriman log melonjak drastis.

4.3. T3 (Bab 3): At-least-once vs exactly-once delivery; peran idempotent consumer.

Dalam menghadapi ketidakpastian jaringan, sistem terdistribusi harus memilih semantik pengiriman pesan yang tepat untuk menjamin reliabilitas. Coulouris et al. (2011, Hal. 174-176) mengklasifikasikan semantik pemanggilan jarak jauh (*remote invocation*) ke dalam beberapa jenis, di mana *at-least-once* dan *at-most-once* merupakan semantik yang paling relevan. Semantik *at-least-once* menjamin bahwa sebuah pesan pasti akan diterima dan dieksekusi setidaknya satu kali, namun memiliki kelemahan bawaan di mana pesan yang sama dapat tereksekusi berulang kali akibat transmisi ulang (*retry*) otomatis saat terjadi gangguan jaringan sementara. Sebaliknya, mencapai semantik *exactly-once* murni sangatlah sulit, bahkan hampir mustahil diwujudkan secara alamiah dalam jaringan yang tidak dapat diandalkan tanpa bantuan penyimpanan status (*stateful*). Untuk menjembatani batasan tersebut, proyek ini membiarkan arsitektur antrean beroperasi dengan semantik pengiriman *at-least-once*, namun merancang penerima akhir (*consumer*) agar bersifat idempoten (*idempotent consumer*). Idempotensi, sebagaimana dijelaskan dalam literatur Coulouris, adalah sifat matematis di mana eksekusi suatu operasi secara berulang akan menghasilkan dampak yang sama persis dengan eksekusi tunggal. Dengan menerapkan pola ini di sisi *worker* agregator, sistem secara aman menelan aliran data yang terduplikasi dari jaringan, dan mendelegasikan penyaringannya kepada pangkalan data untuk mengonversinya menjadi hasil akhir seolah-olah menggunakan semantik *exactly-once*.

4.4. T4 (Bab 4): Skema penamaan topic dan event_id (unik, collision-resistant) untuk dedup.

Keberhasilan implementasi *idempotent consumer* sangat bergantung pada kemampuan sistem untuk secara presisi dan deterministik mengenali identitas setiap pesan yang melintas. Coulouris et al. (2011, Hal. 353) menekankan pentingnya penggunaan Pengenal Unik Global atau *Globally Unique Identifiers* (GUID) dalam sistem terdistribusi. Pengenal ini krusial karena entitas data diciptakan secara independen oleh berbagai layanan klien tanpa adanya otoritas sentralisasi yang

mengatur urutan penomoran. Untuk memenuhi kaidah ini, sistem agregator menerapkan skema penamaan berbasis UUID (*Universally Unique Identifier*) versi 4 sebagai *event_id*, yang kemudian dikombinasikan dengan pengelompokan berdasarkan kategori *topic*. Probabilitas tabrakan data (*collision*) pada UUIDv4 secara matematis sangat mendekati nol, sehingga menjamin bahwa setiap kejadian log yang lahir dari *publisher* mana pun memiliki identitas yang mutlak unik lintas jaringan. Kombinasi pasangan antarkolom *topic* dan *event_id* ini kemudian dipatenkan sebagai kunci pembatas unik (*unique constraint*) pada level skema pangkalan data. Melalui metode identifikasi persisten yang tahan tabrakan ini, pangkalan data mampu mencocokkan setiap pesan yang masuk dengan riwayat pemrosesan sebelumnya, memberikan penolakan tegas terhadap setiap pesan yang membawa pengenalan ganda, dan mengamankan integritas deduplikasi.

4.5. T5 (Bab 5): Ordering praktis (timestamp + monotonic counter); batasan dan dampaknya.

Pengurutan kejadian (*event ordering*) merupakan salah satu tantangan paling fundamental dan kompleks dalam sistem terdistribusi akibat ketiadaan referensi waktu yang absolut. Coulouris et al. (2011, Hal. 595) menguraikan fenomena penyimpangan waktu (*clock drift*), di mana jam fisik pada berbagai komputer berdetak dengan kecepatan yang sedikit berbeda, sehingga menyulitkan sinkronisasi waktu dan penentuan urutan kejadian lintas *node* secara mutlak presisi. Mengingat *Log Aggregator* ini tidak dirancang untuk menangani transaksi perbankan yang menuntut urutan kausalitas ketat seperti pada algoritma waktu logis (*Lamport Logical Clocks*), sistem mengambil pendekatan pengurutan praktis menggunakan stempel waktu fisik (*physical timestamp*) berformat UTC. Stempel waktu ini disuntikkan langsung oleh *publisher* pada saat log tersebut diproduksi. Pendekatan ini memberikan kemudahan bagi tim analitik untuk membaca urutan kejadian secara kronologis kasat mata. Akan tetapi, sesuai dengan batasan teoritis jam fisik, pendekatan ini memiliki dampak arsitektural yang tidak dapat dihindari. Perbedaan sinkronisasi antar-server klien (*clock skew*) berpotensi menciptakan anomali di mana kejadian yang secara faktual terjadi lebih akhir justru tercatat lebih awal di dalam

sistem apabila *server* pengirimnya memiliki jam fisik yang lebih lambat. Dampak pengurutan asinkron ini disadari sepenuhnya dalam desain sistem dan menjadi batasan inheren dalam penentuan urutan kausalitas log lintas layanan.

4.6. T6 (Bab 6): Failure modes dan mitigasi (retry, backoff, durable dedup store, crash recovery).

Dalam lingkungan jaringan yang tidak dapat diandalkan, sistem terdistribusi secara inheren selalu dihadapkan pada ancaman anomali dan interupsi. Coulouris et al. (2011, Hal. 60-61) mengklasifikasikan model kegagalan (*failure models*) ke dalam beberapa kategori utama, di mana kegagalan penghilangan (*omission failures*) akibat pesan yang hilang di jaringan dan kegagalan sistem (*crash failures*) pada proses *server* menjadi ancaman yang paling krusial. Untuk memitigasi kegagalan *omission*, layanan klien atau *publisher* umumnya mengimplementasikan mekanisme pengiriman ulang otomatis (*retry*) yang sering kali disertai dengan jeda waktu penundaan (*backoff*). Akan tetapi, transmisi ulang ini berpotensi menciptakan efek samping berupa tumpukan aliran data ganda apabila kegagalan yang terjadi sebenarnya berada pada fase pengiriman respons jaringan, bukan pada fase penerimaan awal. Untuk mengatasi komplikasi ini, arsitektur *log aggregator* dirancang dengan lapisan penyimpanan deduplikasi yang persisten (*durable dedup store*). Ketika aplikasi mengalami anomali *crash*, kondisi (*state*) dari data log yang telah tervalidasi tidak boleh ikut lenyap. Dengan mengisolasi penyimpanan pangkalan data PostgreSQL ke dalam *volume* fisik yang terlepas dari siklus hidup fana sebuah kontainer Docker, sistem memiliki kemampuan pemulihan pasca-kegagalan (*crash recovery*) yang instan. Segera setelah sistem dihidupkan kembali, rekam jejak deduplikasi masa lalu akan langsung aktif kembali, memberikan jaminan bahwa sisa-sisa pesan yang ditransmisikan ulang oleh klien tidak akan pernah diproses untuk kedua kalinya.

4.7. T7 (Bab 7): Eventual consistency pada aggregator; peran idempotency + dedup.

Kebutuhan akan ketersediaan sistem yang tinggi dan toleransi terhadap penundaan proses memotivasi arsitektur ini untuk melonggarkan batasan seketika dan beralih mengadopsi model konsistensi pada akhirnya (*eventual consistency*).

Berdasarkan penjelasan Coulouris et al. (2011, Hal. 776) mengenai pemeliharaan data dalam sistem skala besar, arsitektur terdistribusi modern sering kali harus menoleransi kondisi di mana replika atau komponen penyimpanan tidak memiliki keadaan data yang sama persis pada milidetik yang sama demi mempertahankan tingkat responsivitas. Dalam konteks proyek *Idempotent Log Aggregator* ini, ketika layanan klien menembakkan log ke arah API, lapisan perantara (*broker*) Redis akan langsung mengembalikan status respons berhasil kepada klien meskipun data log tersebut sebenarnya belum dituliskan secara permanen ke dalam pangkalan data utama. Pendekatan asinkron ini memang menciptakan jendela penundaan (*latency window*), di mana metrik pada *endpoint* analitik statistik belum merefleksikan keberadaan log yang baru saja masuk. Kendati demikian, berkat mekanisme penulisan idempoten dan deduplikasi asinkron yang bekerja secara agresif di latar belakang, sistem memberikan jaminan matematis bahwa seluruh aliran pesan tersebut pada akhirnya akan menyatu (*converge*) menjadi sebuah keadaan pangkalan data yang tunggal, akurat, dan bersih dari redundansi. Melalui pemodelan *eventual consistency* ini, sistem berhasil menyeimbangkan antara kecepatan penerimaan log ekstrem dengan integritas analitik data tingkat tinggi.

4.8. T8 (Bab 8): Desain transaksi: ACID, isolation level, dan strategi menghindari lost-update.

Dalam arsitektur sistem terdistribusi multi-*worker*, potensi bentrokan pembaruan data secara konkuren menjadi tantangan kritis yang harus ditangani secara struktural. Coulouris et al. (2011, Hal. 679) menggarisbawahi pentingnya properti ACID (*Atomicity, Consistency, Isolation, Durability*) dalam merancang transaksi pangkalan data yang tangguh. Salah satu anomali paling berbahaya yang dihindari melalui penerapan tingkat isolasi (*isolation level*) yang tepat adalah masalah pembaruan yang hilang atau *lost update problem*. Fenomena ini terjadi ketika dua proses secara bersamaan membaca status data yang sama, lalu melakukan modifikasi dan penulisan ulang, yang mengakibatkan hasil dari salah satu proses tertimpa dan lenyap tanpa terdeteksi oleh sistem. Untuk mencegah skenario fatal ini pada arsitektur agregator, sistem memanfaatkan mekanisme kontrol transaksi

pangkalan data relasional secara ketat. Dengan memastikan bahwa setiap operasi penyisipan data dan agregasi metrik dibungkus dalam satu blok transaksi yang terisolasi, sistem menjamin bahwa tidak ada dua proses pekerja (*worker*) yang dapat mengganggu alur modifikasi satu sama lain, sehingga integritas log dan statistik tetap terpelihara dengan akurat.

4.9. T9 (Bab 9): Kontrol konkurensi: locking/unique constraints/upsert; idempotent write pattern.

Implementasi tingkat isolasi transaksi dieksekusi melalui strategi kontrol konkurensi yang berfokus pada pencegahan kondisi balapan (*race condition*). Menurut Coulouris et al. (2011, Hal. 692), sistem basis data mengandalkan mekanisme penguncian (*locking*) untuk menyinkronkan akses konkuren terhadap objek data yang sama guna menjaga konsistensi. Daripada membebani lapisan aplikasi (*worker*) dengan logika penguncian manual yang rumit dan rentan terhadap kebuntuan komputasi (*deadlock*), proyek ini mendelegasikan tanggung jawab kontrol konkurensi sepenuhnya kepada pangkalan data melalui pola penulisan idempoten (*idempotent write pattern*). Sistem menerapkan batasan unik (*unique constraint*) pada level skema tabel dan memanfaatkan klausa transaksi isolasi tingkat baris seperti **ON CONFLICT DO NOTHING** (*Upsert*). Pendekatan ini memungkinkan seluruh pekerja yang beroperasi secara paralel untuk berebut memasukkan data log secara agresif tanpa menimbulkan anomali pangkalan data. Sistem penyimpanan secara otomatis akan memberikan kunci pada baris data, menerima penyisipan pertama yang masuk, dan menolak penyisipan duplikat berikutnya secara mulus, menciptakan sebuah mekanisme deduplikasi yang elegan dan berkinerja tinggi.

4.10. T10 (Bab 10–13): Orkestrasi Compose, keamanan jaringan lokal, persistensi (volume), observability.

Ketahanan fungsionalitas dari arsitektur perangkat lunak ini akhirnya disatukan dan dilindungi melalui lapisan orkestrasi kontainer. Coulouris et al. (2011, Hal. 85) menyatakan bahwa langkah perlindungan fundamental dalam sistem terdistribusi adalah mengamankan jaringan melalui penciptaan batas pelindung seperti *firewall*

atau jaringan internal (*intranet*) yang memblokir akses pihak luar yang tidak sah. Prinsip keamanan jaringan lokal ini diwujudkan dengan menggunakan fitur orkestrasi Docker Compose, yang secara otomatis membungkus seluruh komponen—mulai dari *aggregator*, *worker*, perantara Redis, hingga pangkalan data—ke dalam satu jaringan virtual yang terisolasi (*bridge network*). Melalui isolasi ini, akses dari dunia luar hanya diizinkan melalui pintu masuk atau gerbang API yang didefinisikan secara eksplisit, melindungi infrastruktur kritis dari ancaman intervensi langsung. Selain itu, persistensi data dijamin sepenuhnya melalui manajemen *volume* berpenamaan (*named volumes*) yang menjaga integritas penyimpanan fisik pangkalan data meskipun seluruh kontainer mengalami penghentian paksa. Sebagai pelengkap kualitas perangkat lunak berskala produksi, arsitektur ini juga mengintegrasikan fitur observabilitas melalui pencatatan *log* aktivitas internal yang terpusat dan pelaporan metrik kinerja secara seketika (*real-time*). Hal ini memberikan transparansi penuh terhadap kesehatan sistem multi-*worker*, memungkinkan pemantauan yang komprehensif, dan memvalidasi keandalan distribusi beban komputasi di bawah tekanan.

DAFTAR PUSTAKA

Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Sistem terdistribusi: Konsep dan desain* (Edisi ke-5). Pearson Education.